OrthoAI GPU/CUDA Inference Deployment Runbook

Prepared for DevOps | Generated 2026-06-13 15:46

Objective

Move the live inference workload from CPU to CUDA-backed GPU execution while keeping the FastAPI backend/API on CPU. Only the Celery inference worker needs GPU capacity. This reduces user wait time without paying for GPU on web traffic.

Current Production Blocker

The current backend image installs CPU-only PyTorch, so CUDA cannot be used even if the ECS task is placed on a GPU instance. The current Dockerfile includes:

```
RUN pip install --no-cache-dir --index-url https://download.pytorch.org/whl/cpu \
    torch==2.5.1+cpu \
    torchvision==0.20.1+cpu
```

The model runtime already supports CUDA through ORTHOAI_DEVICE=cuda, but the container image and ECS capacity must be changed for live GPU inference.

Target Architecture

- FastAPI backend service remains CPU-based and continues serving demo.orthoai.co/API traffic.
- Inference jobs continue to be queued through Redis/Celery.
- A separate Celery GPU service consumes only inference jobs from a gpu-inference queue.
- The GPU worker uses a CUDA-enabled PyTorch image and requests one physical GPU in ECS.
- The model is preloaded at worker startup to avoid per-request model load latency.

1. Create CUDA Worker Image

Create a separate Dockerfile, for example Dockerfile.gpu. Do not replace the CPU web image unless the whole deployment is intentionally moving to GPU, which is not recommended.

```
FROM nvidia/cuda:12.4.1-cudnn-runtime-ubuntu22.04

ENV PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1 \
    PIP_NO_CACHE_DIR=1 \
    PYTHONPATH=/app \
    ORTHOAI_DEVICE=cuda \
    NVIDIA_DRIVER_CAPABILITIES=compute,utility

RUN apt-get update && apt-get install -y --no-install-recommends \
    python3.11 python3.11-venv python3-pip \
    postgresql-client libpq-dev gcc curl \
    && rm -rf /var/lib/apt/lists/*

RUN useradd --create-home --shell /bin/bash appuser
WORKDIR /app

COPY requirements.txt .
RUN python3.11 -m pip install --upgrade pip
RUN python3.11 -m pip install --index-url https://download.pytorch.org/whl/cu124 \
    torch torchvision
RUN python3.11 -m pip install -r requirements.txt

COPY . .
RUN mkdir -p uploads && chown -R appuser:appuser /app
USER appuser

CMD ["celery", "-A", "app.celery_app", "worker", "-Q", "gpu-inference", "--loglevel=info", "--concurrency=1"]
```

2. Build and Push GPU Image

Use a separate ECR repository or a separate image tag so the backend CPU image and GPU worker image can be promoted independently.

```
ECR_GPU_REPOSITORY=medical-ai-production-celery-gpu
GPU_IMAGE_URI="$AWS_ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com/$ECR_GPU_REPOSITORY:$GITHUB_SHA"

docker build -f Dockerfile.gpu -t "$GPU_IMAGE_URI" .
docker push "$GPU_IMAGE_URI"
```

3. Provision ECS GPU Capacity

- Use ECS on EC2 GPU capacity, not CPU-only Fargate capacity.
- Start with g5.xlarge or g6.xlarge: 1 NVIDIA GPU, 24 GiB GPU memory.
- Use the ECS GPU-optimized AMI so NVIDIA drivers and container runtime are installed.
- Set Auto Scaling Group min=1, desired=1, max=1 or 2 initially.
- Attach the GPU Auto Scaling Group to the existing ECS cluster as a capacity provider.

4. Register GPU Celery Task Definition

The GPU Celery container must reserve a GPU at container definition level.

```
"resourceRequirements": [
  {
    "type": "GPU",
    "value": "1"
  }
],
"environment": [
  { "name": "ORTHOAI_DEVICE", "value": "cuda" },
  { "name": "NVIDIA_DRIVER_CAPABILITIES", "value": "compute,utility" },
  { "name": "PRELOAD_MODEL_RUNTIME", "value": "true" },
  { "name": "DEV_MOCK_INFERENCE", "value": "false" }
],
"command": [
  "celery", "-A", "app.celery_app", "worker",
  "-Q", "gpu-inference", "--loglevel=info", "--concurrency=1"
]
```

5. Route Inference Tasks to GPU Queue

Engineering should add Celery routing so CPU workers do not consume inference tasks.

```
celery_app.conf.update(
    task_routes={
        "app.tasks.inference.run_inference": {"queue": "gpu-inference"},
        "send_otp_email_task": {"queue": "default"},
    },
)
```

6. CUDA Startup Guard

Do not allow silent CPU fallback in production when ORTHOAI_DEVICE=cuda is configured. Add a startup check in the worker/runtime so deployment fails if CUDA is unavailable.

```
        import torch

        if settings.device == "cuda" and not torch.cuda.is_available():
            raise RuntimeError("ORTHOAI_DEVICE=cuda was requested, but CUDA is unavailable")

        logger.info("CUDA available=%s device=%s", torch.cuda.is_available(), runtime.device)
```

7. Deployment Order

- Deploy the CPU FastAPI backend normally.
- Deploy any CPU/default Celery worker only for non-inference tasks such as email.
- Deploy the GPU Celery inference worker on the ECS GPU capacity provider.
- Confirm the ECS task is placed on g5/g6 capacity and has GPU=1 reserved.
- Run one real inference against production S3/model weights.
- Confirm logs show CUDA available and model device cuda.

8. Validation Commands

```
# ECS task placement and container status
aws ecs describe-services --cluster medical-ai-production-cluster   --services medical-ai-production-celery-gpu

# Worker logs should include CUDA/device confirmation
aws logs tail /ecs/medical-ai-production-celery-gpu --follow

# Expected runtime evidence in logs/result JSON
# torch.cuda.is_available() = True
# device = cuda
# model_predict_seconds materially lower than CPU baseline
```

Acceptance Criteria

- GPU Celery ECS task is running on EC2 GPU capacity.
- Container logs show CUDA available and runtime device cuda.
- Inference result JSON contains model device cuda.
- CPU/default Celery workers do not process app.tasks.inference.run_inference.
- model_predict_seconds is materially lower than the CPU baseline.
- API remains responsive while inference is running.

Rollback Plan

- Keep the previous CPU Celery task definition available.
- If GPU worker fails startup, scale GPU service to 0 and restore CPU Celery inference route temporarily.
- Do not roll back database or uploaded cases; inference jobs can be retried once worker capacity is healthy.
- Keep PRELOAD_MODEL_RUNTIME=true after recovery to avoid cold-start waits.

Reference Links

AWS ECS GPU workloads: https://docs.aws.amazon.com/AmazonECS/latest/developerguide/ecs-gpu.html

ECS GPU resourceRequirements API: https://docs.aws.amazon.com/AmazonECS/latest/APIReference/API_ResourceRequirement.html

> Production note: for predictable latency, keep at least one GPU worker warm. Scaling to zero will reintroduce EC2 startup and model-load latency.